KD-Tree Based Smart Robot Object Location System

Understanding KD-Tree and Its Application in AI and Robotics

Presented by Akash Patel

Data Structures and Algorithms II
Professor: Dr. Victor Govindaswamy

Introduction to KD-Tree

- KD-Tree stands for K-Dimensional Tree. It organizes points using their coordinates.
- It is a special binary tree. Each node can connect to a left child and a right child.
- A normal binary search tree usually compares one value. A KD-Tree can work with two or more dimensions.
- For example, the point (5,4) means X = 5 and Y = 4.
- The tree divides points into smaller groups based on location. A search may avoid checking every stored point.
- Applications include robotics, computer graphics, mapping, and location-based searching.

Why Do We Need a KD-Tree?

The warehouse problem

- A warehouse robot works around boxes, shelves, and workstations.
- It may need to find the closest object or check whether an object exists at a location.
- A simple approach checks every object one by one. More objects mean more comparisons.

How the tree helps

- A KD-Tree groups object locations using coordinate comparisons.
- The robot can focus on promising locations and avoid some unnecessary checks.
- This helps when the robot repeatedly asks location questions as it moves.

KD-Tree guides the search instead of always checking every object.

Understanding X and Y Coordinates

- X represents horizontal position. Y represents vertical position.
- Every location is a pair (X,Y). Boxes (4,7) means X = 4 and Y = 7.
- The robot has its own position: (6,5). It is the query point, not one of the four stored objects.
- Coordinates let us compare positions and calculate distances.

Location	X	Y
Boxes	4	7
Shelf B	2	6
Workstation	1	5
Shelf A	8	3
Robot	6	5

How a KD-Tree Organizes Points

Nodes and branches

- A KD-Tree is a binary tree. A node can have a left child and a right child.
- Each node stores an object's name and coordinates.
- Compare the selected coordinate: smaller values move left. Otherwise, move right.
- Our project rejects duplicate coordinate pairs.

Alternating dimensions

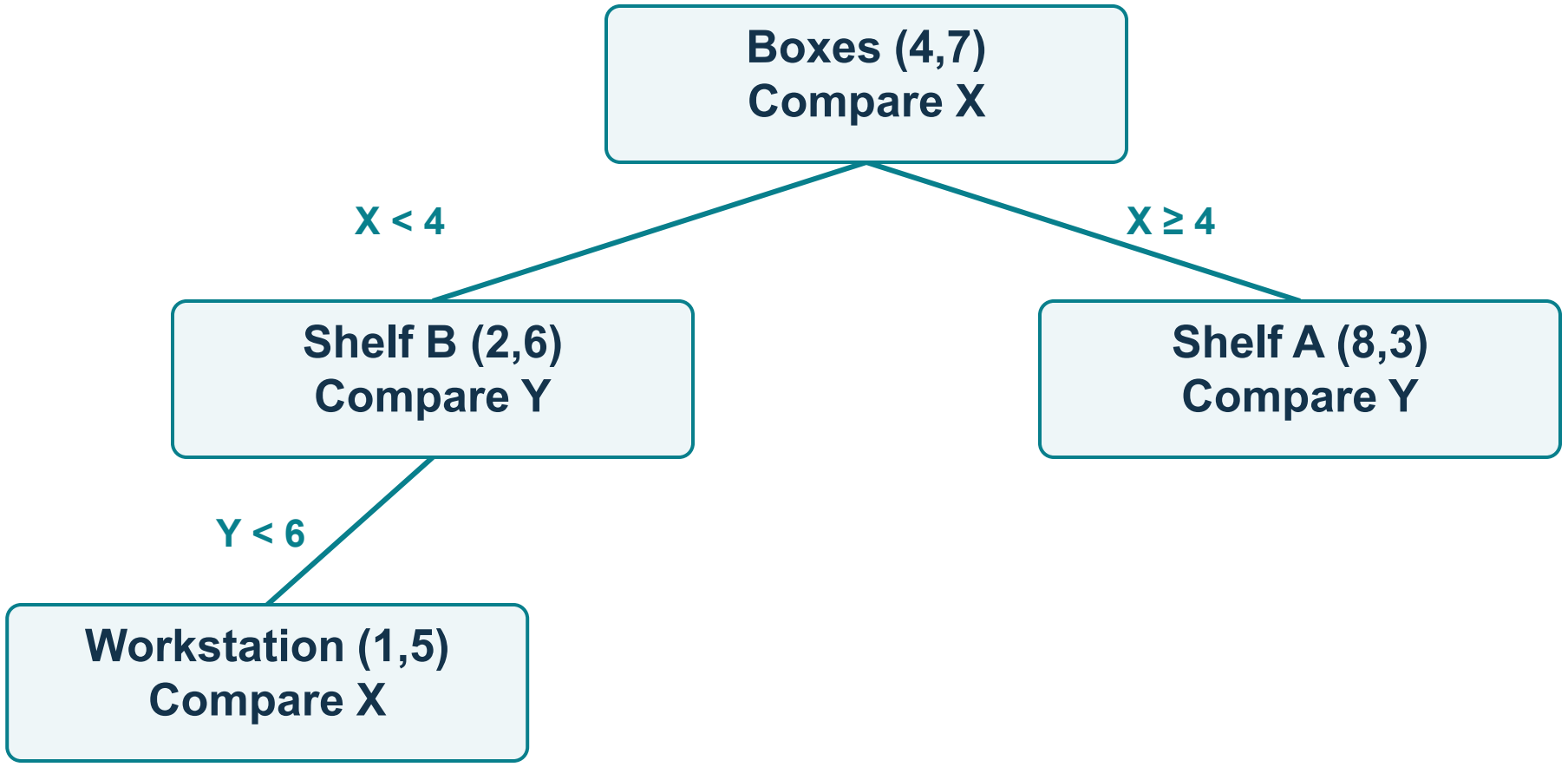
Depth	Compare
0 (root)	X
1	Y
2	X again
3	Y again

Changing dimensions lets the tree organize both horizontal and vertical positions.

Understanding the KD-Tree Structure

- 1. Boxes (4,7) is the first object, so it becomes the root at depth 0.
- 2. Insert Shelf B (2,6). At Boxes, compare X: 2 < 4, so Shelf B becomes the left child.
- 3. Insert Workstation (1,5). At Boxes, X = 1 < 4, so move left. At Shelf B, compare Y: 5 < 6, so insert it on the left.
- 4. Insert Shelf A (8,3). At Boxes, X = 8 > 4, so Shelf A becomes the right child.
- Comparisons alternate by depth: X at the root, Y for its children, then X again. This lets each level use a different dimension.
- The next slide shows the resulting tree. Each connecting line represents a parent–child relationship.

KD-Tree Structure Diagram



- Boxes is the root.
- Shelf B and Shelf A are its children.
- Workstation is Shelf B's left child.
- Branches follow the dimension shown at the parent.

How Insertion Works in a KD-Tree

Finding the empty position

- Check whether the tree is empty. If it is, create the root.
- Otherwise, begin at the root and compare X at depth 0.
- At depth 1 compare Y. Continue alternating X and Y as you go down.
- Move left for a smaller selected coordinate. Otherwise, move right.
- Insert the point when you reach an empty child position.

Example: insert Shelf B

Existing root: Boxes (4,7)
New point: Shelf B (2,6)

At depth 0, compare X:
2 < 4

The left child is empty, so Shelf B becomes Boxes' left child. Duplicate coordinates are rejected.

How Searching Works in a KD-Tree

Example: find Workstation (1,5)

- 1. Start at Boxes (4,7). The coordinates do not match the target.
- 2. Compare X: 1 < 4, so follow the left branch.
- 3. Reach Shelf B (2,6). The coordinates still do not match.
- 4. At this depth, compare Y: 5 < 6, so follow the left branch.
- 5. Reach Workstation (1,5). Both X and Y match, so the object is found.
- If a search reaches an empty child before finding a match, that coordinate pair is not in the tree.

What Is Nearest-Neighbor Search?

- Nearest-neighbor search finds the stored point closest to a given location.
- In our project, the query point is the robot's current position, such as (6,5).
- The robot may ask for the nearest shelf, box, or workstation. We compare distances calculated from their coordinates.
- KD-Tree guides the search toward promising locations and remembers the closest point found so far.
- Sometimes the search must return to an earlier node and check the opposite branch.
- Following only one path can miss a closer point across a dividing boundary. A coordinate comparison alone does not prove which object is closest.

Understanding Distance Calculation

- Euclidean distance is the straight-line distance between two coordinate points.
- Find the X and Y differences, square them, add them, then take the square root.
- Robot = (6,5) and Boxes = (4,7).
- This distance does not account for walls or obstacles. A robot may need a longer route to reach the object.

Euclidean distance

$$d = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$$

$$d = \sqrt{[(4-6)^2 + (7-5)^2]}$$

$$d = \sqrt{[4 + 4]}$$

$$d = \sqrt{8}$$

$$d \approx 2.83 \text{ units}$$

Example: Finding the Nearest Object

Robot position = (6,5)

Stored object	Coordinates	Distance
Boxes	(4,7)	2.83 units
Shelf B	(2,6)	4.12 units
Workstation	(1,5)	5.00 units
Shelf A	(8,3)	2.83 units

- Boxes and Shelf A are equally close. Both have an exact distance of $\sqrt{8}$ units.
- If the program returns one nearest object, either Boxes or Shelf A is a valid result. The distances above are rounded to two decimal places.

How KD-Tree Speeds Up Nearest Search

- A simple search may check every object. A KD-Tree uses coordinate comparisons to guide its search.
- It usually visits the more promising side first, based on the robot’s position relative to the current split.
- It keeps the closest object and its distance found so far.
- Before skipping the opposite branch, it checks whether that branch could contain a closer object.
- If the distance to the split is greater than the best distance, that opposite branch cannot improve the result and may be skipped.
- Skipping a branch this way is called pruning. Otherwise, the search returns and checks that branch.
- Difficult point arrangements or an unbalanced tree can still require checking all objects.

Removing an Object from a KD-Tree

Why removal needs care

- An object may need to be removed when it leaves the warehouse.
- A node may have children below it. Removing it incorrectly can break connections or the coordinate ordering.
- Standard KD-Tree deletion can replace nodes while preserving the tree's rules.

The approach in my project

- I use a simpler rebuild approach rather than standard recursive KD-Tree deletion.
- The program collects the remaining objects and rebuilds the tree without the removed object.
- This is easier to understand and implement, but rebuilding takes additional time.
- The resulting balance depends on the order in which the remaining objects are inserted.

Advantages and Limitations

Advantages

- Organizes points using coordinates.
- Supports exact location searching.
- Supports nearest-neighbor searching.
- Can reduce unnecessary comparisons by skipping irrelevant branches.
- Useful for two-dimensional and three-dimensional location data.

Limitations

- Performance depends on tree balance.
- An unfavorable insertion order can create a long, unbalanced tree.
- Deletion is more complicated than simply removing a leaf in every case.
- In high-dimensional spaces, pruning can become less effective.

Time Complexity of KD-Tree

Operation	Typical / favorable case	Worst case
Insert	O(log n) when balanced	O(n)
Exact search	O(log n) when balanced	O(n)
Nearest-neighbor search	Often efficient in low dimensions	O(n)
Display all objects	O(n)	O(n)
Delete by rebuilding	O(n log n) if reasonably balanced	$O(n^2)$, sequential insertion

- n is the number of stored objects. O(log n) means the work grows slowly as n increases. O(n) means work can grow in proportion to all n objects.
- Rebuilding by repeated insertion can take quadratic time in the worst case. My program does not automatically balance the KD-Tree.

Real-World Applications

- Robotics: search a map of object locations to find nearby items or reference points.
- Computer graphics: organize spatial points for nearby-point queries used in graphics processing.
- Geographic applications: search coordinate data for nearby locations, using a distance model suitable for the map.
- 3D point clouds: organize points from depth sensors or scanners and find nearby points.
- Machine learning: support nearest-neighbor queries when comparing data points, especially in a small number of dimensions.
- Our warehouse example uses the same location-search idea: the robot queries stored objects to find which one is nearby.

How KD-Tree Connects to Our Robot Project

Data and robot position

- The robot works in a simulated warehouse.
- Each object has a name and an (X,Y) coordinate pair.
- A KD-Tree stores the object locations.
- The robot's current location can be updated as the simulation changes.

What the system can do

- Find the nearest stored object to the robot.
- Calculate distance to a selected object.
- Let users add, search, display, and remove objects.
- Demonstrate spatial data structures through a practical example.

This is a simulation, not a real robot navigation system. It does not plan obstacle-free movement.

What I Learned from This Topic

- I previously understood basic binary trees. KD-Tree showed me how a tree can organize multidimensional data.
- I learned why comparisons alternate between X and Y as we move down the tree.
- Following the warehouse points step by step helped me understand insertion and exact searching.
- I learned that nearest-neighbor search must compare distances and sometimes check another branch.
- Understanding removal and nearest-neighbor searching was challenging. Rebuilding made removal easier for me to follow.
- The warehouse example helped me connect the theory to a real-world robotics problem and explain what my simulation can do.

Conclusion

- A KD-Tree is a binary tree that organizes points in two or more dimensions.
- In our two-dimensional example, comparisons alternate between X and Y to choose branches.
- Insertion adds an object at an empty position. Exact search follows comparisons until it finds matching coordinates.
- Nearest-neighbor search compares distances and uses pruning when a branch cannot improve the answer.
- Our project removes an object by rebuilding the tree. Tree balance and insertion order affect performance.
- KD-Tree is useful for location queries in robotics. This project helped me understand spatial data structures through a warehouse simulation.

References

1. GeeksforGeeks — KD Trees in C++

https://www.geeksforgeeks.org/cpp/kd-trees-in-cpp/

2. SSOJet — Implement KD-Tree in C++

https://ssojet.com/dev-guides/implement-kd-tree-in-cpp

3. LeetCode Discuss — KD-Tree Search, Insert, Delete Summary

https://leetcode.com/discuss/post/136595/kd-tree-search-insert-delete-summary-wit-5qzx/

Thank You for Watching!

- That brings me to the end of Part 1 of my presentation on the KD-Tree Based Smart Robot Object Location System.
- Through this project, I learned how a KD-Tree organizes locations using alternating X and Y comparisons. The warehouse example helped me understand insertion, searching, and how a robot can find its nearest object. I also learned why tree balance matters and why nearest-neighbor searching sometimes needs to check another branch.
- I would like to sincerely thank Dr. Victor Govindaswamy for the opportunity to study this topic and explore how data structures connect to AI and robotics.
- In Part 2, I'll demonstrate my C++ program live in Ubuntu and explain how these concepts work through the code and its output.
- Thank you for watching!